Próximos pasos

Corridas para la máquina de 32 GB

Capstone ICS2122 — Grupo 9

7 de noviembre de 2026

En una página

Optimizamos el itinerario semanal de una flota de carga de 19 aviones: qué vuelos hace cada avión, a qué hora, cuánta carga lleva y dónde la transborda. El objetivo es maximizar el margen (ingresos menos costos).

- **Mejor resultado actual: 6.475.055 USD**, validado. Está en la carpeta `data/results/Busqueda_ventanas_desplazar_d2`. Es un 2,6 % más que la heurística inicial (6.310.378).
- Ese resultado se obtuvo en un notebook con 8 GB de RAM y 4 núcleos. Varios métodos que prometen más **no caben** en esa memoria.
- **Tu tarea:** correr 4 búsquedas en tu computador, una por noche, y subir los resultados. Cada paso trae el comando exacto, qué hace, por qué lo corremos y cómo saber si salió bien.
- Todos los comandos están también en `docs/comandos_32gb.txt`. Cópialos desde ahí y no desde el PDF, porque el PDF corta las líneas largas.

Tabla 1: Cómo llegamos al mejor resultado (margen validado, USD).

Etapa	Margen	Aporte
Heurística con grilla de 2 h	6.310.378	—
Heurística con grilla de 1 h y punto de partida	6.336.259	+25.881
MILP integrado en tiempo continuo, con NoRel	6.435.839	+99.580
Búsqueda por vecindarios: un avión a la vez	6.451.322	+15.483
Búsqueda por vecindarios: ventanas de 12 h	6.466.083	+14.761
Ventanas con desplazamiento (agregar o quitar vuelos)	**6.475.055**	+8.972

1. Conceptos que necesitas

Margen validado
El único número que comparamos. Toma los vuelos de una solución, recalcula la carga con un modelo exacto (gap 10^{-6}) y revisa todas las reglas con el validador. Las búsquedas solo aceptan soluciones que pasan el validador.

Heurística
La solución inicial. Carpetas `Heuristica_dt1_iter2` y `Heuristica_dt1_iter3_AND`.

Red espacio-tiempo
La semana se divide en bloques de DT horas (aquí, 1 h). Cada vuelo es un arco entre dos pares (aeropuerto, bloque). Su relajación es ajustada, pero redondea hacia arriba la duración de cada vuelo, así que pierde minutos.

MILP en tiempo continuo
Cada avión tiene posiciones $1, \ldots, 32$ y cada posición lleva un vuelo con su horario exacto. No pierde minutos, pero su relajación es débil (gap de $\sim$30 %).

Vecindario
Una parte de las decisiones que se deja libre mientras el resto queda fijo. Puede ser un avión, un par o un trío de aviones, un operador completo o una ventana de tiempo. Se resuelve el MILP exacto con la carga y los transbordos de *toda* la red, y el cambio se acepta solo si sube el margen validado.

Ventana de tiempo
Libera todos los vuelos que salen entre t_0 y t_1, en todos los aviones a la vez. Permite reorganizar varios aviones juntos; así salió la mejora en la que 9 aviones se reordenaron en MIA.

Desplazamiento
Permite que una ventana gane o pierda hasta D vuelos: los vuelos posteriores mantienen tramo y horario, pero se corren de posición. Aprovecha los aviones que tienen 20–76 h libres en la semana.

NoRel
Heurística de Gurobi que busca soluciones sin resolver la relajación. Sirve cuando la raíz es muy cara, como en nuestro modelo completo.

VNS
Alternar vecindarios: si uno no mejora, se pasa al siguiente; tras una mejora, se vuelve al primero.

2. Por qué hace falta tu computador

Cada método crece con el tamaño de lo que se deja libre. La tabla muestra lo que se estimó con `--estimar`:

Tabla 2: Memoria aproximada de la red espacio-tiempo por vecindario (instancia completa).

Vecindario	DT	Variables	RAM
1 avión	1 h	1,9 M	8 GB (justo)
2 aviones	1 h	3,2 M	16 GB
4 aviones (operadores BRA y PAC)	1 h	5,2 M	32 GB
8 aviones (operador AND)	1 h	10,6 M	64 GB
Todos los aviones	1 h	—	no cabe

En el notebook:

- la red espacio-tiempo de un avión con DT = 1 h pidió más de 5,5 GB en la raíz, así que no se pudo recorrer la flota con pares;
- las ventanas de 24 h no terminaron la raíz en el tiempo dado;
- el MILP integrado corrió con solo 2 hilos y 25 min de NoRel, para no quedarse sin memoria.

Con 32 GB y más núcleos, las tres cosas caben.

3. El plan

Tabla 3: Las corridas, en orden. Corre una a la vez: cada una usa todos los núcleos.

Paso	Duración	Qué hace	Parte de
0	30 min	Instalar y verificar	—
1	noche (12 h)	Red espacio-tiempo, aviones de a uno y de a dos	heurística
2	noche (12 h)	Tiempo continuo con ventanas	resultado del Paso 1
3	noche (12 h)	Vecindarios grandes: ventanas de 24 h, tríos, operadores	mejor actual
4	2 h + noche	MILP integrado (2 veces) y luego el Paso 2 sobre el mejor	heurística
5	10 min	Revisar y subir los resultados	—

Los pasos 1 y 2 van juntos (el 2 parte del 1). Los pasos 3 y 4 son independientes y pueden hacerse en cualquier orden.

Así se ven las dos cadenas que estamos comparando:

- **Rama A** (la que tiene hoy el mejor resultado): heurística → MILP integrado → búsqueda continua. El Paso 4 la repite con más recursos.
- **Rama B** (la que ganó en la instancia reducida R4, con +12,6 %): heurística → red espacio-tiempo → búsqueda continua. Los pasos 1 y 2 la corren por primera vez en la instancia completa.

En R4, la rama B llegó a 868.027 y la A se quedó en 839.809: **el punto de partida decide a qué óptimo local se llega**. Por eso vale la pena correr las dos.

4. Paso 0 — Preparar el computador (una vez)

1. Crea una cuenta en Gurobi con tu correo UC y pide una licencia académica: `https://www.gurobi.com/academia/academic-program-and-licenses/`. Actívala con `grbgetkey <tu-clave>`, conectado a la red UC o por VPN.
2. Clona la rama, instala las dependencias y verifica Gurobi:

```
git clone -b milp-completo-warm-start
    https://github.com/Franconicolai/ICS2122-Grupo-9.git
cd ICS2122-Grupo-9
pip install gurobipy pandas numpy scipy pyyaml python-dotenv highspy
python -c "import gurobipy as gp; gp.Model(); print('Gurobi OK')"
```

3. Deja el computador enchufado. En Windows, el script evita la suspensión por sí solo. En Mac, antepone `caffeinate -i` a cada comando; en Linux, `systemd-inhibit`.
4. Ejecuta todo desde la carpeta `ICS2122-Grupo-9`. En Mac o Linux, cambia `python` por `python3`.

Cómo saber si salió bien: el último comando imprime `Gurobi OK`. Si dice *size-limited license*, falta la licencia académica.

5. Paso 1 — Red espacio-tiempo con grilla de 1 h

Qué hace: recorre los 19 aviones y los 15 pares de aviones más afines. En cada uno libera esos aviones, fija los demás y resuelve la red espacio-tiempo exacta con la carga de toda la red. Si el margen validado sube, acepta el cambio y sigue desde ahí.

Por qué: es la rama B. En R4 esta búsqueda dio +8,6 % sobre la heurística, más que el modelo completo con NoRel (+5,3 %). En la instancia completa nunca se pudo correr: no cabía en 8 GB.

```
python -u busqueda_por_avion.py --corrida data/results/Heuristica_dt1_iter3_AND --dt 1
    --vecindario avion,par --orden uso --max-pares 15 --tiempo 1200 --memoria 24
    --mejora-minima 2000 --vueltas 20 --horas 12 --salida data/results/RamaB_red_dt1
    2>&1 | tee data/results/RamaB_red_dt1.log
```

Qué esperar: hasta 20 min y 24 GB por vecindario; una vuelta completa toma unas 11 h. Al llegar a las 12 h se detiene solo.

Cómo saber si salió bien: el margen final supera 6.345.128, el de la heurística de partida.

6. Paso 2 — Tiempo continuo sobre el resultado del Paso 1

Qué hace: toma los vuelos del Paso 1 y los pule en tiempo continuo, sin el redondeo de la grilla. Alterna ventanas de 12 h, con hasta 2 vuelos más o menos por avión, con vecindarios de un avión y de dos.

Por qué: es la segunda mitad de la rama B. En R4 esta misma cadena llegó a 868.027, el mejor resultado en esa instancia (+12,6 % sobre la heurística).

```
python -u busqueda_por_avion.py --modelo continuo --corrida data/results/RamaB_red_dt1
    --vecindario ventana,avion,par --ventana-h 12 --ventana-paso 6 --desplazar 2
    --desplazar-aviones 10 --desplazar-presupuesto 60 --tiempo 300 --norel 60
    --saltar-sin-cambios --umbral-aceptar 100 --mejora-minima 2000 --orden uso
    --memoria 24 --vueltas 20 --horas 12 --salida data/results/RamaB_continua 2>&1 |
    tee data/results/RamaB_continua.log
```

Qué esperar: la mayoría de las ventanas de 12 h se resuelve con óptimo probado en 1,5–4 min. Una pasada por las 27 ventanas de la semana toma ∼1,5 h.

Cómo saber si salió bien: el margen supera el del Paso 1. Si además supera **6.475.055**, es el nuevo mejor resultado.

7. Paso 3 — Vecindarios grandes desde el mejor actual

Qué hace: parte de la mejor solución (6.475.055) y prueba vecindarios más grandes que los que cabían en el notebook: ventanas de 24 h, tríos de aviones y operadores completos.

Por qué: 6.475.055 es un óptimo local para todos los vecindarios que probamos. Para salir de él hacen falta vecindarios más grandes, y esos necesitan memoria.

```
python -u busqueda_por_avion.py --modelo continuo --corrida
    data/results/Busqueda_ventanas_desplazar_d2 --vecindario ventana,trio,operador
    --ventana-h 24 --ventana-paso 12 --desplazar 2 --desplazar-aviones 10
    --desplazar-presupuesto 60 --tiempo 900 --norel 120 --saltar-sin-cambios
    --umbral-aceptar 100 --mejora-minima 2000 --orden uso --memoria 24 --vueltas 20
```

```
    --horas 12 --salida data/results/Grandes_desde_mejor 2>&1 | tee
    data/results/Grandes_desde_mejor.log
```

Qué esperar: hasta 15 min por vecindario. Es posible que no mejore: en R4, liberar 3 de los 4 aviones no sacó a la búsqueda de su óptimo local.

Cómo saber si salió bien: cualquier margen sobre 6.475.055 es una mejora. Si no hay ninguna, el resultado igual sirve: muestra que el óptimo local resiste vecindarios grandes.

8. Paso 4 — MILP integrado en tiempo continuo

Qué hace: resuelve el modelo completo con todos los aviones libres, partiendo de la heurística, usando NoRel para buscar soluciones. Después, al mejor resultado se le aplica la búsqueda del Paso 2.

Por qué: es la rama A, la que dio el mayor salto (+99.580). En el notebook corrió con solo 2 hilos y 25 min de NoRel. NoRel tiene azar: desde otra heurística dio 6.422.235 en vez de 6.435.839. Por eso conviene correrlo más tiempo y más de una vez.

```
python -u warm_start_milp.py --corrida data/results/Heuristica_dt1_iter2 --tiempo 3600
    --norel 3000 --hilos 8 --salida data/results/MILP_integrado_h8 2>&1 | tee
    data/results/MILP_integrado_h8.log
python -u warm_start_milp.py --corrida data/results/Heuristica_dt1_iter2 --tiempo 3600
    --norel 3000 --hilos 6 --salida data/results/MILP_integrado_h6 2>&1 | tee
    data/results/MILP_integrado_h6.log
```

Qué esperar: ∼1 h cada una. El número de hilos cambia el camino de NoRel, por eso se corre con 8 y con 6.

Después: si alguno supera **6.435.839**, corre la búsqueda continua sobre él durante una noche. Si el mejor fue el de 6 hilos, cambia `h8` por `h6` en `--corrida`:

```
python -u busqueda_por_avion.py --modelo continuo --corrida
    data/results/MILP_integrado_h8 --vecindario ventana,avion,par --ventana-h 12
    --ventana-paso 6 --desplazar 2 --desplazar-aviones 10 --desplazar-presupuesto 60
    --tiempo 300 --norel 60 --saltar-sin-cambios --umbral-aceptar 100 --mejora-minima
    2000 --orden uso --memoria 24 --vueltas 20 --horas 12 --salida
    data/results/MILP_integrado_continua 2>&1 | tee
    data/results/MILP_integrado_continua.log
```

9. Paso 5 — Revisar y subir

1. Revisa la última parte de cada `.log`:
   - Pasos 1, 2, 3 y 4b (`busqueda_por_avion.py`) terminan con `Margen validado: A -> B USD`, donde A es el margen de partida y B el final. No imprimen «Validador: OK», porque cada solución aceptada ya pasó el validador.
   - Paso 4a (`warm_start_milp.py`) termina con `Margen validado con transbordos: ...` y `Validador: OK`.
2. Para confirmarlo en cualquier paso, abre `indicadores_kpi.json` en la carpeta de salida: debe decir `"validador_ok": true`.

3. En la carpeta de salida, `progreso.csv` tiene una fila por vecindario. La columna `aceptado` marca las mejoras, y `mejor_margen` muestra cómo fue subiendo el margen.

4. Sube los resultados a la misma rama, nunca a `main`:

```
git add data/results/RamaB_* data/results/Grandes_desde_mejor*
    data/results/MILP_integrado*
git commit -m "resultados maquina 32 GB"
git pull
git push
```

5. Avisa al grupo el margen final de cada paso.

10. Las opciones de los comandos

Opción	Qué controla
`--corrida`	Carpeta de la solución de partida.
`--salida`	Carpeta donde se guarda la mejor solución; se actualiza con cada mejora.
`--modelo continuo`	Usa el MILP en tiempo continuo; sin esta opción, la red espacio-tiempo.
`--dt`	Horas por bloque de la red espacio-tiempo.
`--vecindario`	Vecindarios a alternar: `avion`, `par`, `trio`, `operador`, `ventana`.
`--tiempo`, `--norel`	Segundos de Gurobi por vecindario, y cuántos de ellos se dedican a NoRel.
`--memoria`	GB máximos para Gurobi; con 32 GB, 24 deja espacio al sistema.
`--hilos`	Núcleos de Gurobi (0 = todos).
`--horas`, `--vueltas`	Detienen la búsqueda tras esas horas o vueltas completas.
`--mejora-minima`	Una mejora menor que esto (USD) no hace volver al primer vecindario.
`--umbral-aceptar`	Mejora mínima (USD) para aceptar un cambio; evita cambios casi equivalentes.
`--saltar-sin-cambios`	Salta un vecindario que ya falló, mientras su región siga igual.
`--ventana-h`, `--ventana-paso`	Ancho de la ventana y cuánto avanza cada vez (h).
`--desplazar`	Vuelos que cada avión puede ganar o perder dentro de la ventana.
`--desplazar-aviones`	Cuántos aviones pueden desplazarse (los con más horas libres).
`--desplazar-presupuesto`	Tope total de vuelos que se corren; sin él, el modelo explota.
`--orden uso`, `--max-pares`	Recorre primero los aviones más usados; limita cuántos pares se prueban.

11. Si algo falla

Síntoma	Qué hacer
Model too large for size-limited license	Falta la licencia académica: repite el Paso 0.1.
Out of memory o el PC se congela	Cambia `--memoria 24` por `--memoria 16`. Si sigue, en el Paso 1 quita `,par`.
Se cortó (luz, reinicio, Ctrl+C)	La mejor solución ya está guardada en `--salida`. Repite el comando con `--corrida` igual a esa carpeta.
Una noche sin mejoras	Es un resultado válido: súbelo igual y pasa al paso siguiente.

12. Qué no vale la pena correr

- **Ramificar el modelo completo hasta el óptimo:** su relajación es débil (gap de 42–47 % en R4), y más memoria no lo arregla.
- **Benders:** es correcto, pero no converge en la práctica.
- **Una segunda pasada del MILP integrado después de la búsqueda continua:** empeora el margen con transbordos.
- **Grillas de 30 min:** aportaron ∼0,1 %.

13. Respaldo en la literatura

Cada método que usamos tiene una base estándar en la literatura:

- **Modelo integrado** (Paso 4). En aerolíneas de carga, la selección de vuelos, las rotaciones y el ruteo de la carga dependen entre sí, y planificarlos por etapas pierde margen. Derigs et al. proponen modelos que los deciden juntos [4, 3].
- **Red espacio-tiempo** (Paso 1). Es el modelo clásico de asignación de flota, como flujo multiproducto en una red expandida en el tiempo [6].
- **Tiempo continuo después de la grilla** (pasos 2–4). Discretizar el tiempo puede excluir las mejores soluciones; Boland et al. muestran que el óptimo en tiempo continuo se alcanza refinando la discretización solo donde hace falta [1]. Eso explica por qué pulir en tiempo continuo la solución de la grilla sube el margen.
- **Búsqueda por vecindarios exactos** (todos los pasos). Liberar una parte de la solución, fijar el resto y reoptimizarla es una búsqueda en vecindarios grandes, LNS [10, 9]. Cuando cada vecindario se resuelve con un MILP sobre un subconjunto de binarias, es un esquema de tipo *fix-and-optimize* [7], emparentado con los vecindarios definidos con un MILP, como RINS [2].
- **Alternar vecindarios** (avión, par, trío, ventana). Es la idea de la búsqueda de vecindad variable, VNS [8].
- **NoRel con punto de partida.** Gurobi documenta esta heurística como útil cuando la relajación de la raíz es muy cara [5].

Nuestra evidencia propia: en R4, la búsqueda por vecindarios ganó al modelo completo (+8,6 % contra +5,3 %). Eso respalda usar LNS en un modelo cuya relajación es débil.

Referencias

[1] N. Boland, M. Hewitt, L. Marshall y M. Savelsbergh (2017). The continuous-time service network design problem. *Operations Research*, 65(5), 1303–1321. doi:10.1287/opre.2017.1624

[2] E. Danna, E. Rothberg y C. Le Pape (2005). Exploring relaxation induced neighborhoods to improve MIP solutions. *Mathematical Programming*, 102(1), 71–90. doi:10.1007/s10107-004-0518-7

[3] U. Derigs y S. Friederichs (2013). Air cargo scheduling: integrated models and solution procedures. *OR Spectrum*, 35(2), 325–362. doi:10.1007/s00291-012-0299-y

[4] U. Derigs, S. Friederichs y S. Schäfer (2009). A new approach for air cargo network planning. *Transportation Science*, 43(3), 370–380. doi:10.1287/trsc.1090.0282

[5] Gurobi Optimization (2026). *Gurobi Optimizer Reference Manual*, parámetro NoRelHeurTime. `https://docs.gurobi.com/projects/optimizer/en/current/reference/parameters.html`

[6] C. A. Hane, C. Barnhart, E. L. Johnson, R. E. Marsten, G. L. Nemhauser y G. Sigismondi (1995). The fleet assignment problem: solving a large-scale integer program. *Mathematical Programming*, 70, 211–232. doi:10.1007/BF01585938

[7] S. Helber y F. Sahling (2010). A fix-and-optimize approach for the multi-level capacitated lot sizing problem. *International Journal of Production Economics*, 123(2), 247–256. `https://econpapers.repec.org/RePEc:eee:proeco:v:123:y:2010:i:2:p:247-256`

[8] N. Mladenović y P. Hansen (1997). Variable neighborhood search. *Computers & Operations Research*, 24(11), 1097–1100. doi:10.1016/S0305-0548(97)00031-2

[9] D. Pisinger y S. Ropke (2010). Large neighborhood search. En M. Gendreau y J.-Y. Potvin (eds.), *Handbook of Metaheuristics*, 2.ª ed., 399–419. Springer. doi:10.1007/978-1-4419-1665-5_13

[10] P. Shaw (1998). Using constraint programming and local search methods to solve vehicle routing problems. En *Principles and Practice of Constraint Programming (CP98)*, LNCS 1520, 417–431. Springer. doi:10.1007/3-540-49481-2_30